

ERGALICS STUDIO

Ergalics Studio

产品介绍

浏览器中的科学计算工作站

一款完全运行于浏览器标签页中的专业科学计算环境：交互式数据探索、WebGPU 实时计算、沙箱化插件生态与四种面向不同用户的工作模式，整合在同一个网页应用里。数据保存在本机浏览器中，打开网页即用，关掉即走。

四种工作模式 · 一条从零代码到真代码的路径

标准

拖入数据
即刻看图

流程

区块搭线
数据流管线

积木

类 Scratch
编程启蒙

代码

Python
真 CPython

开源项目 · MIT 协议

目 录

一、项目背景与设计目标	03
1.1 科学计算工具链的三重门槛	03
1.2 五条设计目标	03
1.3 四类用户，四种模式	04
1.4 项目现状	04
二、技术架构	08
2.1 总体分层	08
2.2 贯穿架构的三条设计主线	09
2.3 一条渲染管线服务全部模式	10
三、功能全景	10
3.1 内置插件生态	10
3.2 流程模式区块体系	11
3.3 数据格式支持	11
3.4 统计、绘图与可复现能力	12
四、应用场景	12
五、创新点	19
六、运行与获取	20

一 项目背景与设计目标

1.1 科学计算工具链的三重门槛

安装与环境门槛高。传统桌面软件（如 MATLAB、Origin）体积庞大、授权昂贵，一台新电脑从安装到能用往往要以小时计；Python 生态虽然免费开放，但要让初学者独立配好解释器、虚拟环境、数值库与绘图库，仍然是一件容易劝退的事。环境问题消耗的，本应是投入在数据与问题本身上的注意力。

教学路径存在断层。从积木式编程过渡到真实代码，中间缺少一款能把“拖拽数据、搭建流程、编写代码”统一在同一界面里的工具。学生在一个工具里学会拖积木，换到另一个工具里面对黑漆漆的终端，两次学习之间没有衔接，已经建立的直觉难以迁移。

数据安全与离线可用。科研与教学数据常涉及隐私或涉密，上传云端在很多场景下不可接受。纯浏览器方案天然做到数据不出本机，同时保留接近桌面软件的交互体验；配合浏览器对 WebGPU、WebAssembly 与 Web Worker 的支持，“重计算发生在网页里”已经从口号变成了现实。

Ergalics Studio 的回答是：把整个工作台——包括 Python 运行时、GPU 计算与插件生态——全部搬进浏览器标签页，做到“打开网页即用、关掉即走、数据不出本机”。

1.2 五条设计目标

1. **零安装，打开即用。**全部能力随网页交付，标准模式下把数据文件拖进窗口就能看到可视化结果。
2. **数据不出本机。**项目与数据存储浏览器 IndexedDB 中，可导出为 .clproj 文件离线流转，不存在云端副本。
3. **一条学习曲线走到底。**四种模式面向同一份项目数据，从看到图、到搭流程、到拼积木、到写代码，能力渐进而不换工具。
4. **计算必须是真计算。**GPU 内核是真实的 WGSL 计算着色器，Python 是真正的 CPython，原生核心由 Rust 编译为 WebAssembly；每一个内核都有对应的单元测试与端到端数值一致性校验。
5. **开放可扩展。**插件契约清晰、市场目录内建、第三方插件默认运行在 Worker 沙箱中，配合完整的开源代码与持续集成，生态可以安全地生长。

1.3 四类用户，四种模式

产品为不同背景的用户提供了渐进式的四种工作模式。四种模式面向同一份项目数据，彼此共享同一套数据语义与渲染后端，切换模式时逻辑原样保留：

模式	面向人群	核心体验
标准	只想快速看图的人	拖入数据文件，自动识别格式并路由到匹配插件，即刻看到可视化
流程	数据分析初学者	用 37 个内置区块搭出可视化数据流管线，拓扑排序执行，逐节点查看输出
积木	编程启蒙与教学	类 Scratch 积木编辑器，唯一入口是绿色"运行时"帽子区块，30 余种积木完全可脚本化
代码	真实脚本用户	Monaco Python 编辑器，由 Pyodide Web Worker 提供真正的 CPython 运行时，带 REPL 控制台与变量面板

1.4 项目现状

项目处于积极开发中，核心闭环已端到端可用并有测试覆盖。当前基线：



- **40 个内置插件**：30 个核心/科学插件随启动自动加载，10 个趣味/工具插件按需加载。
- **四种工作模式**全部可用；积木、流程、代码三种模式经由共享中间表示（IR）双向互转。
- **WebGPU 实时计算**覆盖粒子、N-Body 引力、格子 Boltzmann 流体、波动方程、直方图、热力图、点云等内核，均带 CPU 回退。
- **科学计算子系统**：统计内核、科研二进制 I/O（HDF5、NetCDF、FITS、Zarr、Parquet）、出版级绘图导出、可复现性内核已全部落地。
- **质量保障**：46 个测试文件、417 个单元测试全部通过；10 套 Playwright 端到端套件驱动真实浏览器验证；持续集成自动执行。
- **后续里程碑**：插件市场的包签名与第三方安装管线；代码模式的 R 运行时（webR）。

1.3 四类用户，四种模式（展开）

标准模式是产品的"前门"。工作台采用经典的四区布局：顶栏负责项目与模式切换，左侧是项目树与插件列表，中央视口承载插件渲染，右侧是与所选插件实时绑定的参数面板。把文件拖进窗口，宿主会通过文件格式检测（Rust 核心按魔数识别，辅以扩展名）自动路由到匹配的插件，例如拖入 .xyz 文件直接得到点云渲染。

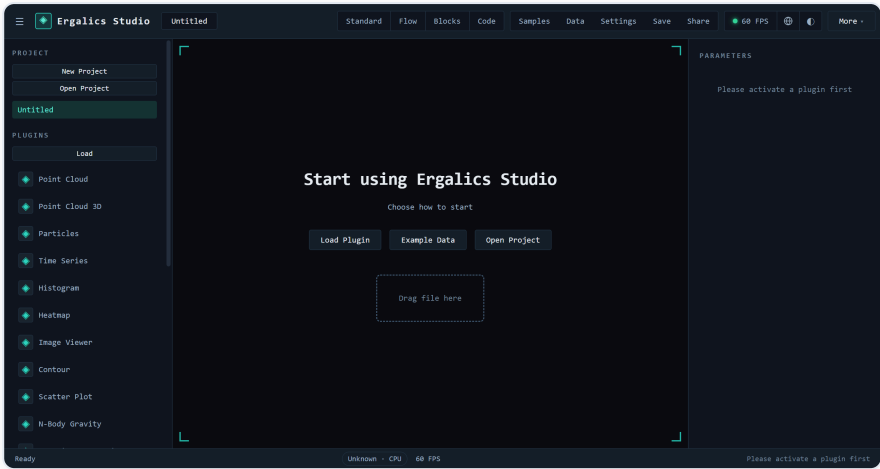


图 1 标准模式：左侧项目与插件列表，中央视口，右侧响应式参数面板

流程模式把分析过程显式化为一张有向无环图。用户从区块目录拖出数据源、变换、过滤、数学、统计与可视化区块，连线成管线；编译器先做结构校验（端口匹配、必填参数、类型检查）与拓扑排序，执行器再按序执行并对每个节点做增量缓存，改动一个参数只重算受影响的下游节点。每个区块的输出都可以就地预览。

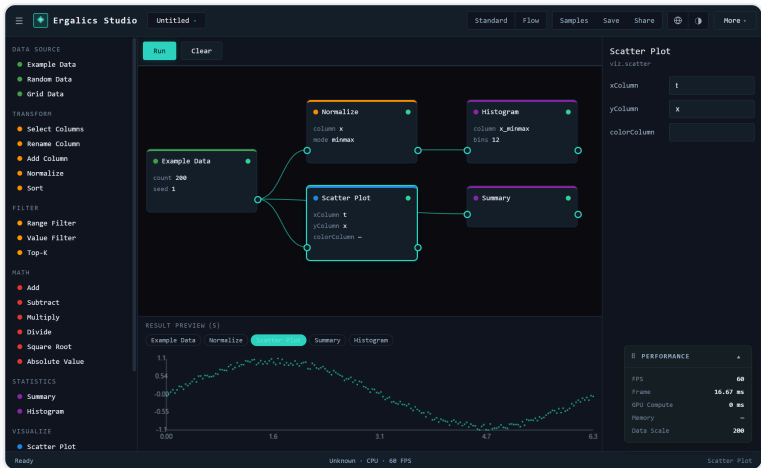


图 2 流程模式：可视化数据流管线，节点输出就地预览

积木模式面向编程启蒙。类 Scratch 的画布上，绿色"运行时"帽子区块是脚本的唯一入口，数据加载、变量、循环、数学运算与绘图积木拼在其下；点击运行后，宿主把积木图编译为共享中间表示（IR），由内置解释器逐节点执行。积木名称、提示与下拉选项均已本地化，5 个内置示例程序开箱即玩。

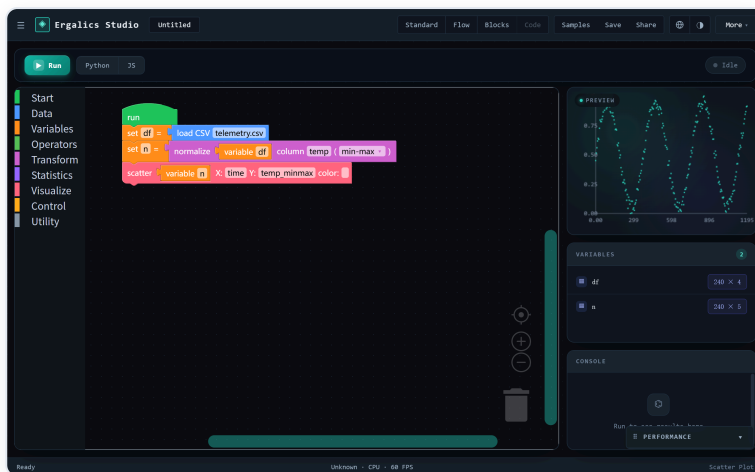


图 3 积木模式："运行时"帽子区块启动脚本，加载遥测数据、归一化并绘图

代码模式面向真实脚本用户。Monaco 编辑器之外，侧栏提供 REPL 控制台与变量面板；Python 代码运行在 Pyodide Worker 中，是货真价实的 CPython。代码里调用的 studio 装载、生成、归一化与绘图接口，与积木模式完全一致；仓库还在 examples/code 目录提供了 9 个可直接运行的示例程序，从蒙特卡洛求圆周率到信号分析一应俱全。

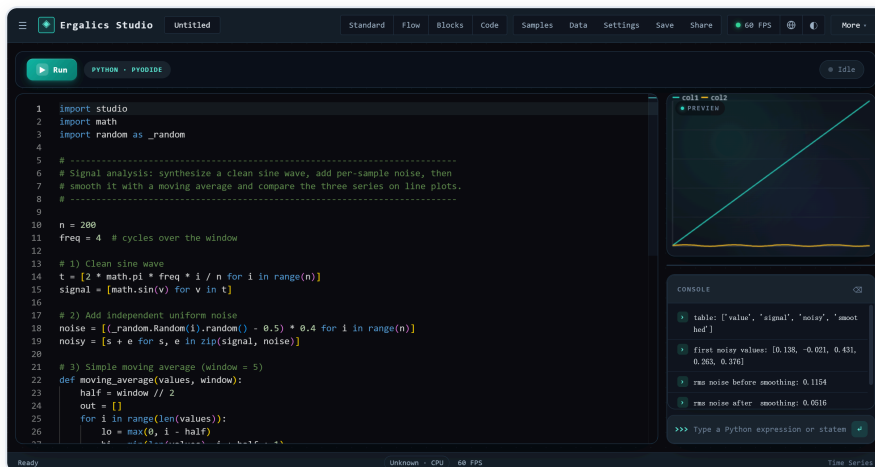


图 4 代码模式：Monaco 编辑器与基于 Pyodide 的 CPython 运行时

1.3 四类用户，四种模式（展开）

三个脚本模式共享同一份 IR：流程里搭好的管线可以变成积木，积木可以生成 Python 代码，代码模式下编辑的 studio 调用也能解析回积木图。一次编辑，三种表达，由专门的往返互转单元测试兜底。



示意图 1 三种脚本模式经由共享 IR 双向互转

为什么这样设计模式体系

把四种模式放进同一个应用，而不是做成四个工具，直接收益是**上下文零损耗**：数据、项目状态、插件渲染与参数面板在模式切换时原样保留，学生不必重新"搬家"。更深一层，共享 IR 让"同一个逻辑的三种表达"成为可以随时切换的视图，而不需要任何导出导入动作——教师可以先用流程讲结构，再切到积木讲控制流，最后切到代码讲语法，全程不离开当前项目。

模式之间的能力边界也是刻意设计的：标准模式刻意隐藏一切编程概念，只保留"数据进、图像出"；流程模式引入节点、连线与参数，但不出现变量与控制流；积木模式引入变量、循环与顺序执行的程序语义；代码模式则暴露完整的 Python 语言与 Pyodide 生态。难度梯度平滑，且每一级都有真实的生产力，而非玩具演示。

技术架构

2.1 总体分层

整体为"React 界面层 + 状态与核心服务层 + 插件运行时与原生核心 + 浏览器底座"的多层结构。所有层级运行在同一浏览器页面内，通过明确的契约通信，低层从不反向导入高层。

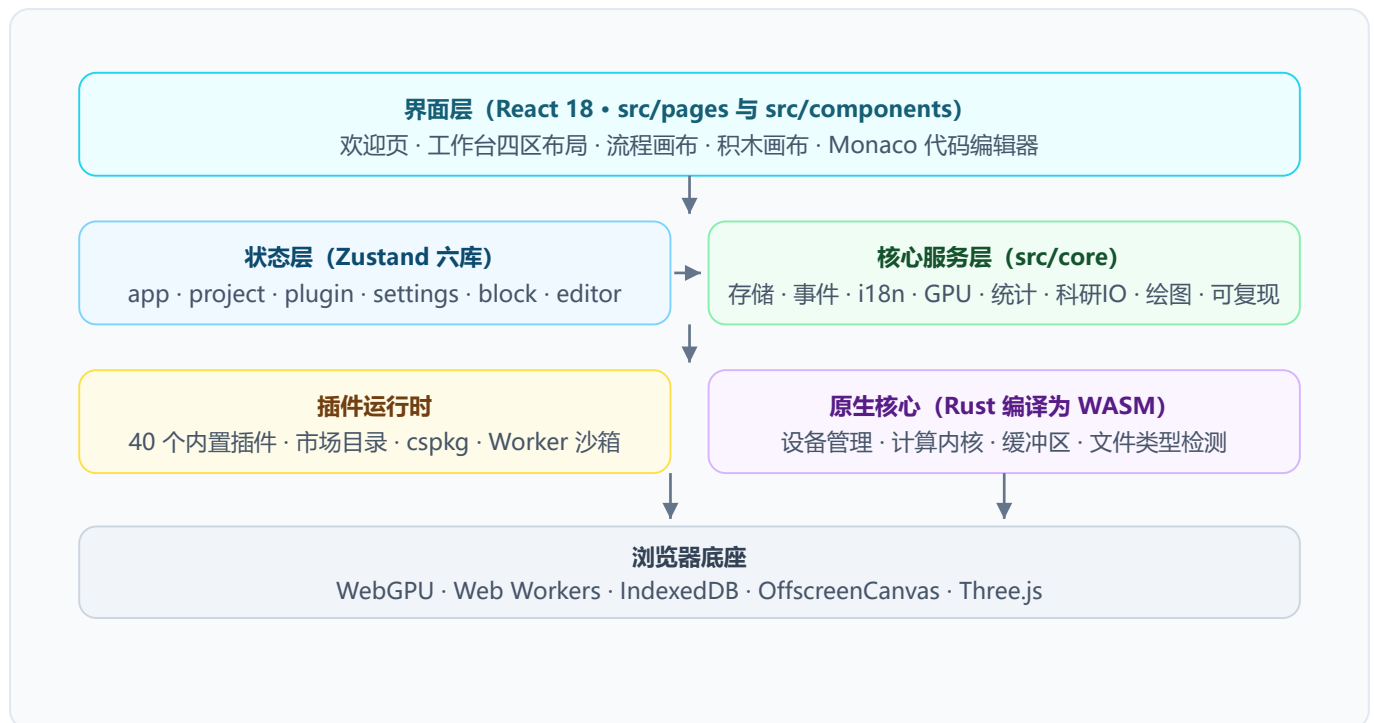


示意图 2 分层架构：低层从不反向导入高层，插件契约是宿主与第三方代码之间唯一的共享词汇表

React 界面层由欢迎页、工作台与三种模式编辑器组成，组件只与状态库和插件契约打交道，不含业务算法。状态层用六个 Zustand 库分别管理应用壳、项目、插件、设置、区块系统与编辑器状态，互相之间通过事件总线协作而非直接引用。

2.1 总体分层（各层职责）

- **核心服务层**（src/core）：存储（IndexedDB 封装）、事件总线、国际化（中英双语）、主题、性能上报、文件格式检测、3D 场景管理、沙箱与 cspkg 加载等横切能力，以及统计内核、科研 I/O、绘图引擎、可复现性内核四个科学计算子系统。
- **插件运行时**（src/plugins）：内置插件的注册表与生命周期、市场目录、cspkg 包加载器与 Worker 沙箱。
- **原生核心**（native 目录，Rust）：编译为 WebAssembly 后向 JavaScript 暴露设备管理、缓冲区与计算内核抽象，并承担文件类型魔数检测。
- **浏览器底座**：WebGPU、Web Workers、IndexedDB、OffscreenCanvas 与 Three.js，是所有能力最终落地的平台。

2.2 贯穿架构的三条设计主线

宿主与插件之间只有一个契约。每个插件实现统一的 Plugin 接口（初始化、激活、渲染、参数读写、数据加载、3D 场景渲染等），并从宿主获得一个 PluginApi 句柄用于本地化、状态上报、性能上报、通知与文件访问。第三方插件的入口代码默认运行在 Web Worker 沙箱中，仅通过类型化 RPC 协议与宿主通信，画布渲染经 OffscreenCanvas 转移完成；.cspkg 包在加载时进行清单校验（id 格式、入口路径穿越防护、沙箱枚举）。

一条渲染管线服务全部模式。无论是流程模式的可视化区块、积木模式解释器调用的 studio.plot，还是代码模式 Python 里的 studio.plot 调用，最终都汇入同一个渲染桥接，落到同一个散点图、直方图插件上。2D 插件共享同一个 canvas 视口；3D 插件按需懒创建宿主管理的 Three.js 场景，并在 2D 插件激活时自动隐藏，保证两种视口互不渗透。

GPU 计算双引擎。Rust 核心编译为 WebAssembly，向 JavaScript 暴露设备管理、缓冲区与计算内核的完整抽象；当 WASM 模块不可用时，宿主侧服务自动改走原生 WebGPU API；两者皆不可用时插件回退到 CPU 实现，行为一致。端到端测试会对 GPU 与 CPU 的计算结果做数值一致性校验（误差约 2 乘以 10 的负 6 次方）。

2.3 一条渲染管线服务全部模式

三种编辑模式产生的绘图请求汇入同一个渲染桥接，由共享的插件渲染器出图，因此"换一种表达方式"看到的是同一张图。



示意图 3 三种编辑模式汇入同一渲染桥接

功能全景

3.1 内置插件生态

30 个核心/科学插件按能力分为六组，随启动自动加载；插件无论内置还是第三方，加载、渲染、参数与数据访问走的是同一条契约路径。市场目录（src/plugins/marketplace.ts）按科学、趣味、工具三类组织，核心插件自动加载、趣味插件手动加载的两级策略保持首屏轻量。

分组	插件	说明
基础可视化	散点、折线（时间序列）、柱状、气泡、直方图、热力图、等值线、图像查看、点云（2D 与 3D）、粒子	覆盖最常见的工程与科学出图需求；2D 插件共享同一 canvas 视口，3D 点云走 Three.js 场景
高级统计图	箱线、小提琴、误差带、QQ 图、平行坐标、桑基、矩形树图、网络图、雷达/极坐标、蛋白质互作网络	面向统计教学与多变量探索，网络类插件内置力导向布局
物理仿真	N-Body 引力、格子 Boltzmann 流体、波动方程、双摆	数据驱动的仿真引擎，WGSL 计算内核加速，带 CPU 回退
交互实验室	电磁场、光学实验、结构力学	元件可直接拖动，实时求解并可视化
机器学习	AI 训练器	四类模型浏览器内训练，实时损失曲线
地理可视化	GeoJSON 地图	离线矢量地图与分级设色，内置中国省份示例

10 个趣味/工具插件按需加载：Mandelbrot 与 Julia 集浏览器、Spirograph 摆线艺术、Lissajous 曲线、Game of Life 元胞自动机、Harmonograph 谐振绘图、调色板探索器、科赫雪花、巴恩斯利蕨、烟花粒子与 Truchet 瓷砖。它们既是课堂演示的素材，也是插件 API 的活文档。

3.2 流程模式区块体系

37 个内置区块分七类，构成流程模式的"词汇表"。区块元数据（名称、描述）内建中英双语；编译器负责结构校验、端口匹配、类型检查与拓扑排序，执行器以节点为粒度做增量缓存与失效重算。

类别	数量	区块
数据源	4	示例数据、随机数据、网格数据、文件导入
变换	5	选择列、重命名列、添加列、归一化、排序
过滤	3	范围过滤、数值过滤、Top-K 选取
数学	6	加、减、乘、除、平方根、绝对值（支持与标量或另一列运算）
统计	11	摘要统计、直方图分箱、单样本/独立两样本/配对 t 检验、单因素方差分析、Mann-Whitney U 检验、卡方检验、相关分析、Cohen's d 效应量、多重比较校正
绘图	4	折线、散点、直方图、柱状（出版级绘图引擎渲染）
可视化	4	散点、折线、直方图、二维点云（经渲染桥接走插件出图）

示例管线（11 个 .clproj 项目文件）位于 examples/projects 目录，覆盖信号分析、随机分布、网格散点、范围过滤、双管线、Top-K、分箱统计与归一化等主题。

3.3 数据格式支持

类别	格式	说明
常规数据	CSV、JSON、XYZ、DAT、PNG、GeoJSON	散点、表格、网格、图像与矢量地图
科研二进制	HDF5、NetCDF、FITS、Zarr、Parquet	由纯 TypeScript 实现的科研 I/O 子系统解析，统一调度器按魔数分派
项目文件	.clproj	工程自有格式，lz-string 压缩，IndexedDB 存储并支持导出分享
插件包	.cspkg	flate 打包的 ZIP，加载时校验清单并落入沙箱

格式检测由 Rust 原生核心按魔数识别，辅以扩展名匹配；识别结果驱动标准模式的插件自动路由。

3.4 统计、绘图与可复现能力

四个纯 TypeScript 科学计算子系统支撑产品的"科学"属性，全部为纯函数实现、Node 环境可测：

- **统计内核** (src/core/stats)：描述统计、分布函数与假设检验，是流程模式 11 个统计区块和摘要数据表的计算后端。
- **科研 I/O** (src/core/io)：单一调度器按文件魔数分派到各格式解析器，支持 HDF5、NetCDF、FITS、Zarr、Parquet 的导入。
- **绘图引擎** (src/core/plot)：纯 TypeScript 的出版级绘图内核，输出 SVG 与 PDF，为流程模式的绘图区块与导出需求提供一致的计算与版式。
- **可复现性内核** (src/core/repro)：带种子的随机数、运行清单（种子、版本、输入哈希、图哈希）与 DAG 转 Python 导出，让一次分析可以在别处原样重放。
- **运行日志导出** (src/core/logger)：把会话内的事件序列落为可下载文件，便于问题回溯与教学演示复盘。

四 应用场景

4.1 科学与工程数据可视化

覆盖统计分析与工程绘图的常见图型：散点、折线、直方图、热力图、等值线、箱线图、小提琴图、误差带、平行坐标、桑基图、矩形树图、QQ 图、气泡图、雷达图、网络图等；数据格式上除 CSV、JSON、XYZ 外，还支持 HDF5、NetCDF、FITS、Zarr、Parquet 等科研领域常用的二进制格式导入，可服务地学、天文、生物等学科的数据探索需求。下图是等值线插件渲染的双高斯标量场。

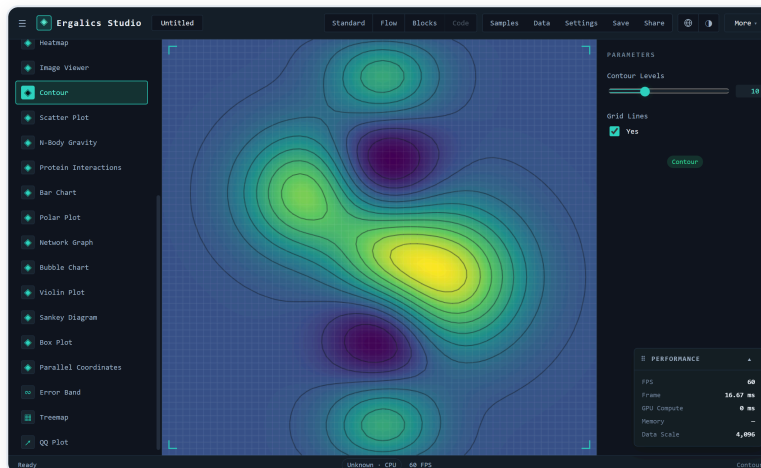


图 5 等值线插件：双高斯标量场，渐变底色叠加等值线

4.2 物理仿真与教学演示

内置多个数据驱动的仿真引擎，全部"初始为空、绝不伪造默认场景"，数据既可来自内置示例也可来自用户文件：

插件	演示内容
N-Body 引力	4096 体的三维全对引力直接求和，WGSL 内核加速
格子 Boltzmann 流体	D2Q9 通道流绕障碍物，展示卡门涡街与机翼绕流
波动方程	高斯脉冲、双源干涉、双缝衍射场景，leapfrog 内核积分
双摆	RK4 积分配合初值仅差 0.001 弧度的"混沌幽灵摆"，直观展示混沌
电磁场	可拖动电荷在库仑力与洛伦兹力共同作用下的回旋运动
光学实验	薄透镜成像、三棱镜折射与色散，元件可在画布上直接拖动
结构力学	铰接桁架实时承重，杆件按轴力着色，超载断裂直至垮塌

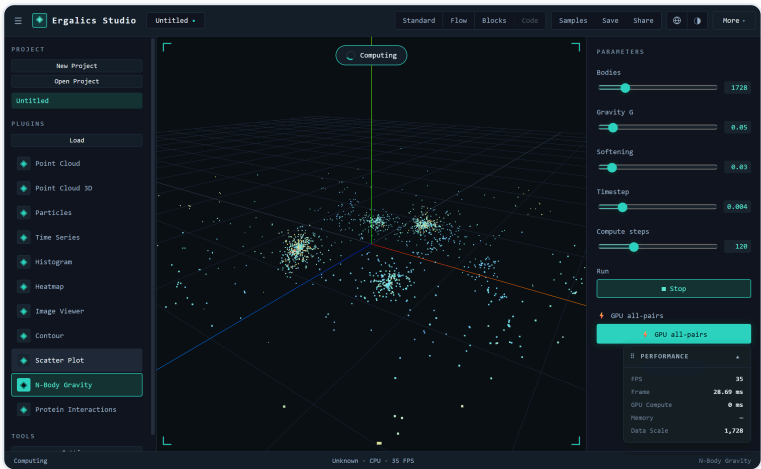


图 6 N-Body 引力：4096 体的环形星环环绕中心质量（三维，WGSL 全对内核）

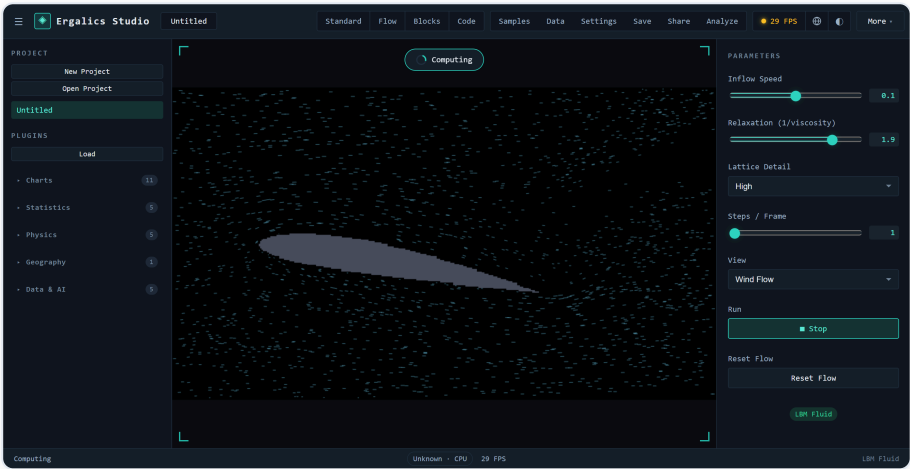


图 7 格子 Boltzmann 流体：绕机翼障碍物的通道流

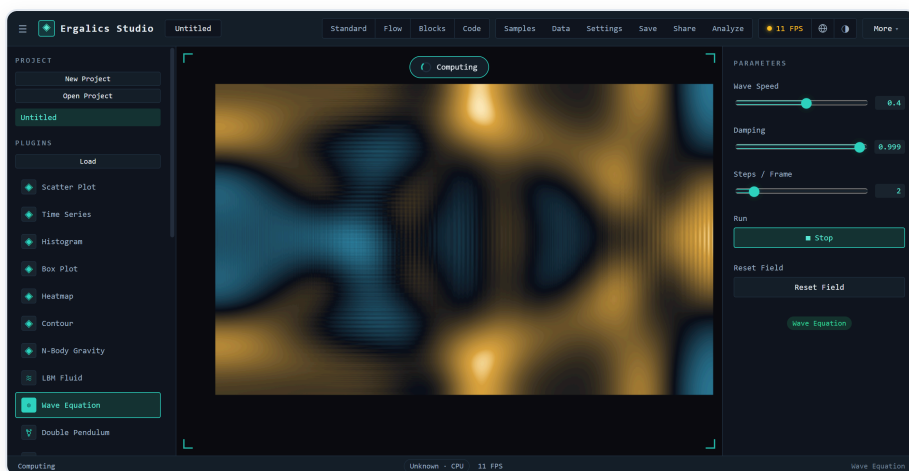


图 8 波动方程：双缝衍射干涉场景，leapfrog WGSL 内核积分

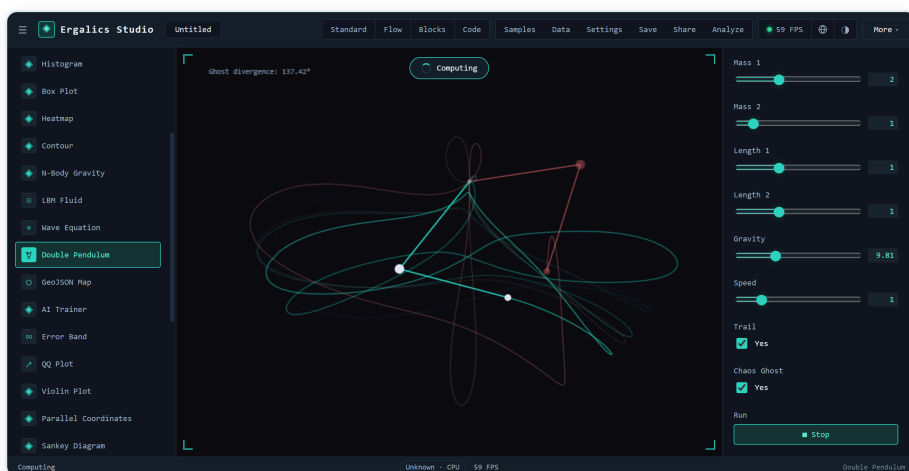


图 9 双摆：主摆与初值仅差 0.001 弧度的混沌幽灵摆迅速分道扬镳

这两个场景共同展示了"计算必须是真计算"的分量：波动方程在 GPU 上以有限差分逐步积分，双摆用四阶龙格-库塔法求解刚性方程组，混沌不是预设的动画，而是数值积分的自然结果——把初值改回一致，两条轨迹就会完全重合。

4.2 物理仿真与教学演示（交互实验室）

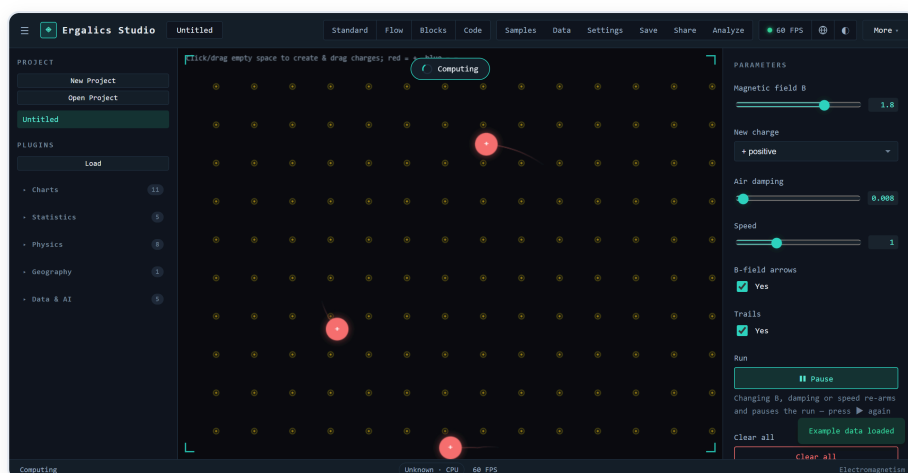


图 10 电磁场实验室：三个正电荷在均匀磁场中回旋，轨迹线开启

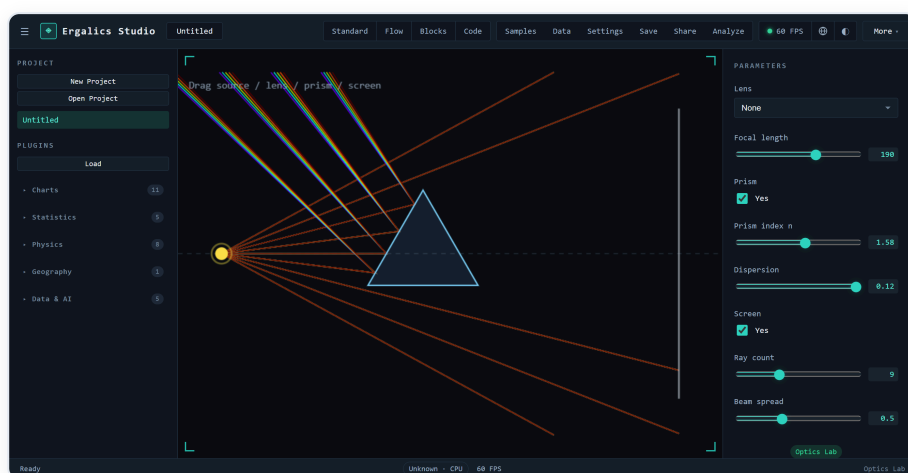


图 11 光学实验室：白光束经三棱镜色散为光谱，汇聚于右侧光屏

三个交互实验室的共同特点是“元件可拖动、方程在背后”：电荷拖近磁场增强回旋半径，透镜与光屏拖动立即改变成像，桁架加载重物实时看到轴力重新分配。参数探索与物理直觉建立同步发生。

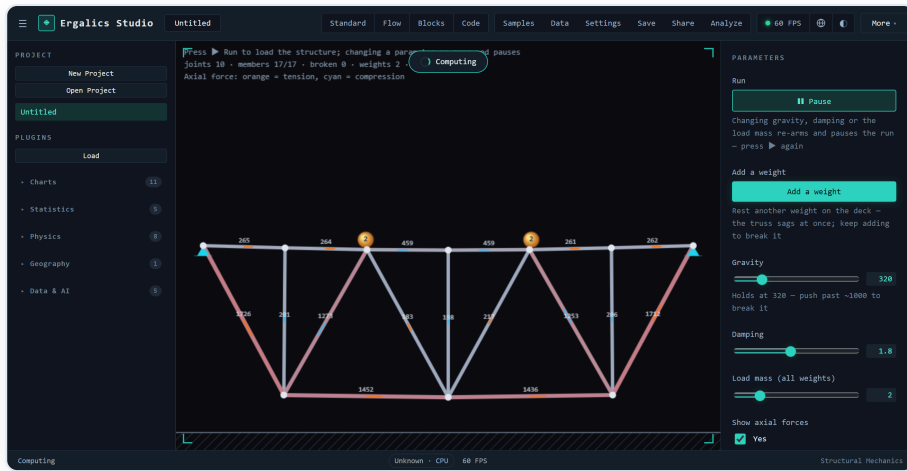


图 12 结构力学：桥面桁架承载两个重物，杆件按轴力着色，橙为拉、青为压



图 13 蛋白质互作网络：力导向布局，网络组件度量实时计算

结构力学插件在杆件超载时会依次断裂直至整体垮塌，把"安全系数"从公式变成看得见的过程；蛋白质互作网络则演示了力导向布局与连通分量度量，可作为生物信息学入门的可视化教具。

4.3 编程与计算思维教学

四种模式构成一条从"零代码"到"真代码"的渐进路径：初学者先在标准模式拖数据看图，再到流程模式理解数据流与统计概念（t 检验、方差分析、相关分析等 11 个统计区块），继而进入积木模式写出第一段带变量与循环的"程序"，最后在代码模式直接写 Python。三个脚本模式共享同一份 IR，切换模式时逻辑原样保留；仓库附带的 11 个流程示例项目、5 个积木示例与 9 个 Python 示例，构成可直接布置的练习素材库。

4.4 浏览器内机器学习入门

AI 训练插件基于 TensorFlow.js，支持线性回归、非线性神经网络、逻辑回归与卷积神经网络四类模型，画布上方实时绘制损失曲线，下方按模型切换散点加拟合线、二维决策边界或 MNIST 数字识别网格。训练样本内置（线性、三次加正弦、双高斯分类、200 张 MNIST 子集），TF.js 依赖在首次点击训练时才懒加载，避免拖慢启动。

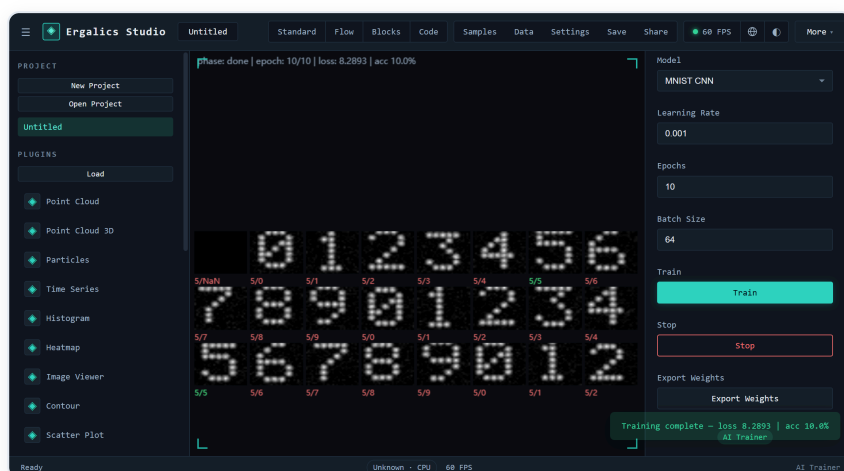


图 14 AI 训练器：MNIST CNN 训练，网格展示预测值（绿）与真值（红）

4.5 地理可视化与算法艺术

离线 GeoJSON 分级设色地图支持 Albers（中国）、Web 墨卡托与等距圆柱投影，内置中国省份示例，可承接人口、面积等统计字段的专题制图需求，全程无需联网请求任何地图服务。



图 15 GeoJSON 地图：Albers（中国）投影下的中国省份分级设色

算法艺术方向的 10 个趣味插件（Mandelbrot 与 Julia 集、科赫雪花、巴恩斯利蕨、Spirograph、Lissajous、Harmonograph、Truchet 瓷砖、烟花粒子、元胞自动机、调色板探索器）覆盖了分形、迭代函数系统、参数曲线与随机结构等主题，适合数学课堂的兴趣延伸，也展示了插件 API 在纯图形场景下的完整能力。

五 创新点

1. 四模式同源架构。市面上的工具要么只做拖拽可视化，要么只做代码编辑。Ergalics Studio 用一份共享 IR 打通积木、流程与代码三种范式，并配备三模式往返互转的专门单元测试；代码模式的缓冲区内容还能解析回 IR，使“换一种表达方式理解同一逻辑”成为一键操作。对教学而言，这意味着同一条管线可以用三种难度梯度反复讲解，而无需任何导出导入。

2. 零安装的全栈科学计算环境。CPython (Pyodide Worker)、Rust/WASM 原生核心、WebGPU 计算与 IndexedDB 持久化全部运行在浏览器内，数据不出本机，同时保持接近桌面软件的体验。这不仅是部署便利，更改变了“教学机房的软件审批、学生自带电脑的环境差异”这类现实约束的性质：所有人打开的是同一个 URL，运行的是同一份环境。

3. 真实 GPU 计算加优雅降级。计算管线不是演示性质的：从 Rust 核心暴露的缓冲区与内核抽象，到粒子、N-Body、流体、波动方程等真实 WGSL 内核，再到端到端测试中 GPU 与 CPU 结果的数值一致性校验（误差约 2 乘以 10 的负 6 次方），每一层都可用、可测；无 WebGPU 环境自动回退 CPU，行为一致。

4. 严格数据驱动的仿真插件。所有仿真插件初始为空，重置只重放已加载数据，杜绝“伪造默认场景”。这一约束保证了演示与真实数据的统一性，也倒逼插件把物理参数做成可调项，而不是把结论画死在界面里。

5. 沙箱化插件生态。第三方插件默认运行于 Web Worker，通过类型化 RPC 访问宿主，画布渲染经 OffscreenCanvas 转移完成；.cspkg 包在加载时进行清单校验（id 格式、入口路径穿越防护、沙箱枚举）。市场目录按科学、趣味、工具分类，核心插件自动加载、趣味插件手动加载的两级策略保持首屏轻量。

6. 可复现性与工程化质量。可复现性内核提供带种子的随机数、运行清单（种子、版本、输入哈希、图哈希）与 DAG 转 Python 导出；统计内核与科研 I/O 均为纯函数实现、Node 环境可测；417 个单元测试加 10 套端到端测试构成持续回归网，在持续集成中自动执行。

六 运行与获取

- **在线演示：**部署于 GitHub Pages（见仓库 README 徽章链接），打开即用，支持最新版 Chrome 与 Edge。
- **本地开发：**Node.js 20 以上，安装依赖后启动开发服务器即可；构建原生核心需要 Rust 工具链与 wasm32 目标，但前端在缺失 WASM 模块时会优雅降级，不影响界面与纯 TS 子系统。
- **文档站点：**仓库内 VitePress 工作区提供架构、四种模式、插件开发与测试指南，与技术文档目录互补。
- **开源协议：**MIT，欢迎通过 Issue 与 Pull Request 参与贡献。

路线图：插件市场的包签名与第三方安装管线、代码模式的 R 运行时（webR）已在规划中；更多统计区块、仿真内核与可视化插件将随版本迭代持续加入。